Monte-Carlo Methods for Reinforcement Learning

Abstract

This topic explores Monte Carlo methods in reinforcement learning, focusing on their application for estimating value functions and discovering optimal policies without requiring a model of the environment. It covers both on-policy and off-policy prediction and control, emphasizing the use of importance sampling to handle data from different policies. By learning from sampled trajectories, students will understand how to estimate values and optimize policies, preparing them for more advanced RL algorithms.

Learning Objectives

- Reproduce and implement First-Visit Monte Carlo Prediction for on-policy value estimation
- Reproduce and implement Monte Carlo Exploring Starts (MC ES) for on-policy control
- Reproduce and implement Monte Carlo with ε-greedy exploration for on-policy control
- Reproduce and implement Off-Policy Monte Carlo Prediction using importance sampling
- Reproduce and implement Off-Policy Monte Carlo Control
- Understand the difference between on-policy and off-policy learning and when to use each
- Explain why MC methods are model-free and how they learn from complete episodes
- Explain the role of exploring starts and ε-greedy in ensuring convergence guarantees

Motivation for Monte Carlo Methods

Why do we need Monte Carlo Methods?

Dynamic Programming methods (as covered in the previous lecture on Dynamic Programming require **complete knowledge of the environment model**, specifically:

- The transition function $p(s'|s, a)$ - probability of reaching state s' when taking action a in state s
- The reward function $r(s, a, s')$ - immediate reward received for transitions

The Reality Problem: In most real-world reinforcement learning scenarios, we **do not have access** to these model parameters. We can only:

- Observe states s_t
- Take actions a_t
- Receive rewards r_t
- Observe next states s_{t+1}

Monte Carlo Solution: Monte Carlo methods are essentially **model-free variants of Dynamic Programming** that learn value functions and policies directly from experience data - sequences τ of (s_t, a_t, r_t, s_{t+1}) tuples.

Key insight: Instead of using the model to compute expected values, we **sample trajectories** and use the actual returns to estimate value functions.

Overview: Monte Carlo Methods and Their Dynamic Programming Analogies

This notebook covers **four key Monte Carlo methods** and their relationship to Dynamic Programming approaches:

Monte Carlo Method	Dynamic Programming Analogy	Key Difference
1. MC On-Policy Prediction	Policy Evaluation	Uses sampled returns instead of expected values from model
2. MC On-Policy Control	Policy Iteration	Improves policy using sampled Q-values, needs exploration
3. MC Off-Policy Prediction	Policy Evaluation (different policy)	Learns about target policy using data from behavior policy
4. MC Off-Policy Control	Policy Iteration (different policy)	Optimizes target policy using data from behavior policy

Core Analogies:

- **Prediction Methods** (1 & 3): Like DP Policy Evaluation, but estimate $V^\pi(s)$ or $Q^\pi(s,a)$ from sample trajectories $\tau = \{(s_0, a_0, r_0), (s_1, a_1, r_1), \ldots\}$
- **Control Methods** (2 & 4): Like DP Policy Iteration, but alternate between:
  - **Evaluation**: Estimate Q-values from samples
  - **Improvement**: Update policy greedily w.r.t. estimated Q-values

Key Innovation: All methods replace the model-dependent expectation operator with **empirical averaging** over sampled returns, enabling learning in unknown environments.

Monte Carlo On-Policy Prediction

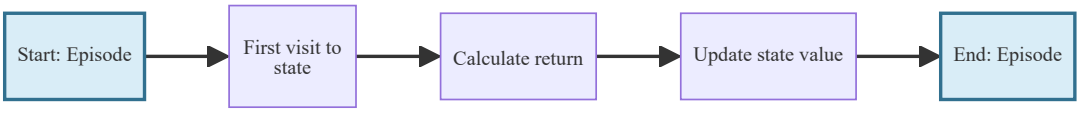
Motivation: We need a way to estimate the value function $V^\pi(s)$ of a given policy π without knowing the environment model (transition probabilities and reward function). On-policy prediction learns from episodes generated by following the same policy we want to evaluate.

What it does: Estimates how good each state is under the current policy by averaging actual returns obtained from that state.

How it works:

1. **Generate episodes**: Follow policy π to collect complete episodes $\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_T)$
2. **Calculate returns**: For each state visited, compute the actual return $G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots$
3. **Average returns**: Estimate $V^\pi(s)$ as the average of all returns observed from state s
4. **First-visit vs Every-visit**: Either average returns from first visits only, or from all visits to each state

First-visit MC method



Algorithm: First-Visit Monte Carlo Prediction

Input: π (policy to evaluate)
Output: $V^{\pi}(s)$ (value function for all states)

1. Initialize $V(s)$ arbitrarily for all $s \in \mathcal{S}$
2. Initialize Returns$(s) \leftarrow$ empty list for all $s \in \mathcal{S}$
3. Repeat forever:
  4. Generate episode following $\pi : s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, r_T$
  5. $G \leftarrow 0$
  6. For $t = T-1, T-2, \ldots, 0 :$
    7. $G \leftarrow \gamma G + r_{t+1}$
    8. If $s_t \notin \{s_0, s_1, \ldots, s_{t-1}\}$: (\texttt \tiny This is the first-visit condition!)
      9. Append G to Returns(s_t)
      10. $V(s_t) \leftarrow$ average(Returns(s_t))

Convergence of first-visit MC: First-visit MC produces independent, identically distributed estimates of the state value, and by the law of large numbers, the average converges to the expected value as the number of first visits increases (assuming a stochastic policy or a stochastic environment).

In [1]:
```python
import importlib
import helper_functions
importlib.reload(helper_functions)
from helper_functions import *


def monte_carlo_prediction(env_name='FrozenLake-v1',
                                    num_episodes=1000,
                                    gamma=0.95,
                                    initial_value='zeros',
                                    plot_interval=100,
                                    show_animation=True,
                                    max_episode_steps=50,
                                    is_slippery=False,
                                    policy_fn=None,
                                    first_visit=True):
    """
    First-Visit Monte Carlo Prediction Algorithm

    This implementation follows the pseudocode algorithm step-by-
step:
    1. Initialize V(s) arbitrarily for all s ∈ S
    2. Initialize Returns(s) ← empty list for all s ∈ S
    3. Repeat forever (or for num_episodes)
    4. Generate episode following π
    5. G ← 0
    6. For t = T-1, T-2, ..., 0:
    7. G ← γG + R_{t+1}
    8. If S_t ∉ {S₀, S₁, ..., S_{t-1}}: (First-visit condition)
    9. Append G to Returns(S_t)
    10. V(S_t) ← average(Returns(S_t))

    Args:
        env_name: Gymnasium environment name
        num_episodes: Number of episodes to run
        gamma: Discount factor
        initial_value: 'zeros', 'random', or array of initial val
ues
        plot_interval: Show plot every N episodes (if show_animat
ion=True)
        show_animation: Whether to show animated plots
        max_episode_steps: Maximum steps per episode
        is_slippery: Whether environment has stochastic transitio
ns
        policy_fn: Policy function (if None, uses random policy)

    Returns:
        tuple: (value_function, returns, episode_stats)
            - value_function: Final estimated value function
            - returns: Dictionary of returns for each state
            - episode_stats: Dictionary with statistics
    """
    import numpy as np
    import gymnasium as gym

    # Create environments
```

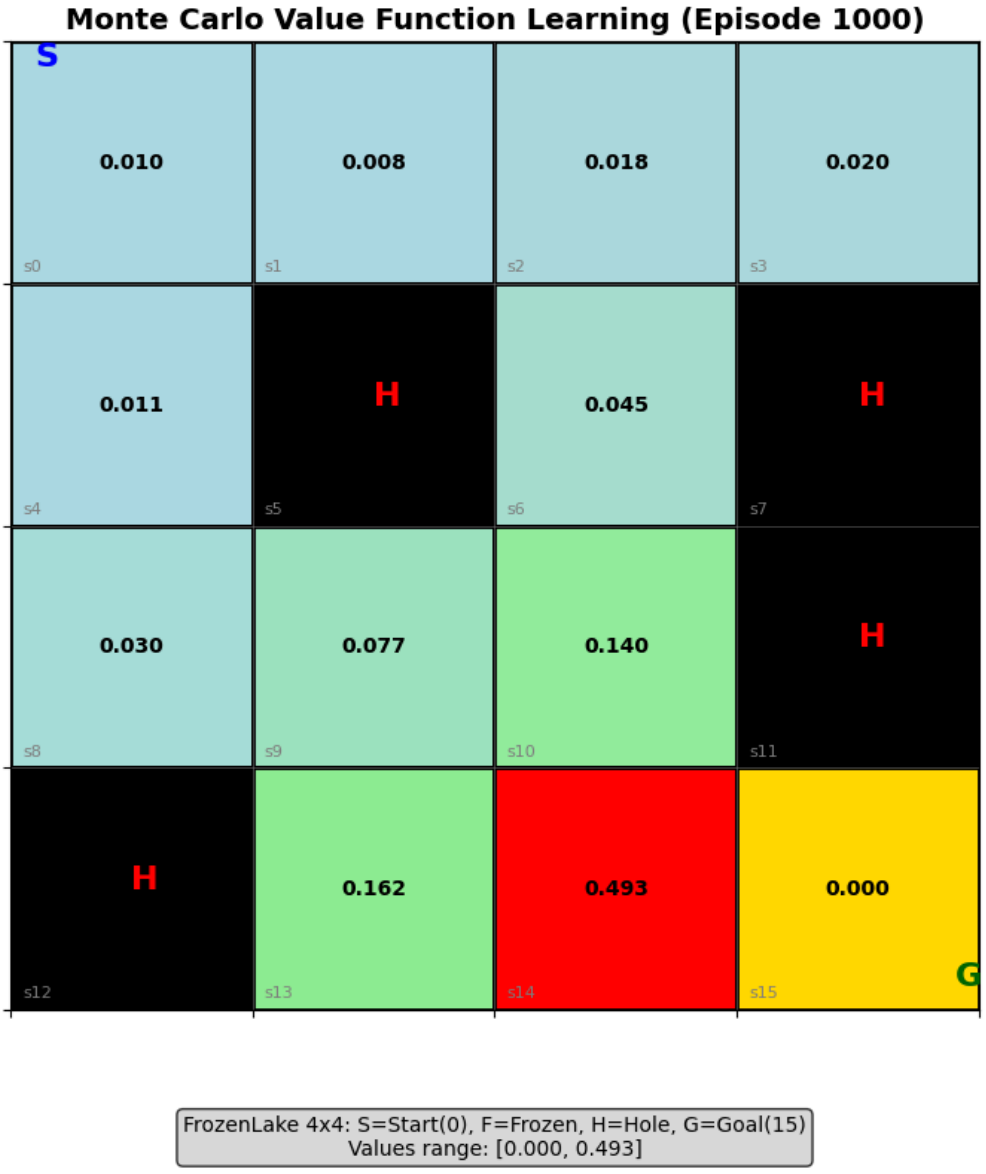
```
    if show_animation:
        rendered_env = gym.make(env_name, is_slippery=is_slipper
y, render_mode="human",
                                max_episode_steps=max_episode_step
s, reward_schedule=(1, 0, 0))
    hidden_env = gym.make(env_name, is_slippery=is_slippery, rend
er_mode="ansi",
                          max_episode_steps=max_episode_steps, rew
ard_schedule=(1, 0, 0))

    # Use hidden environment as default
    env = hidden_env

    # Step 1: Initialize V(s) arbitrarily for all s ∈ S
    if initial_value == 'zeros':
        value_function = np.zeros(env.observation_space.n)
    elif initial_value == 'random':
        value_function = np.random.rand(env.observation_space.n)
    else:
        value_function = np.array(initial_value)

    # Step 2: Initialize Returns(s) ← empty list for all s ∈ S
    returns = {state: [] for state in range(env.observation_spac
e.n)}

    # Define default policy π to evaluate
    if policy_fn is None:
        def policy_fn(state):
            """Simple random policy for demonstration"""
            return np.random.choice(env.action_space.n)

    # Statistics tracking
    episode_stats = {
        'episode_lengths': [],
        'episode_returns': [],
        'states_visited': []
    }

    print("First-Visit Monte Carlo Prediction")
    print("=" * 40)
    print(f"Environment: {env_name}")
    print(f"Episodes: {num_episodes}, γ = {gamma}")
    print(f"Initial values: {initial_value}")

    # Show initial value function
    if show_animation:
        update_mc_value_function_plot(value_function, "Initial Va
lue Function", episode=0)
        time.sleep(1)

    # Step 3: Repeat forever (or for num_episodes)
    for episode in range(num_episodes):
        # Use rendered environment occasionally for visualization
        if show_animation and episode % plot_interval == 0:
            print(f"Episode {episode}/{num_episodes}")
            current_env = rendered_env if show_animation else hid
den_env
```

```
        else:
            current_env = hidden_env

        # Step 4: Generate episode following π: S₀, A₀, R₁, S₁, A
₁, R₂, ..., S_{T-1}, A_{T-1}, R_T
        episode_data = []
        state = current_env.reset()[0]
        done = False
        total_reward = 0

        # Generate complete episode
        while not done:
            action = policy_fn(state)
            next_state, reward, done, _, _ = current_env.step(act
ion)
            episode_data.append((state, action, reward))  # Store
(S_t, A_t, R_{t+1})
            total_reward += reward
            state = next_state

        # Track episode statistics
        episode_stats['episode_lengths'].append(len(episode_dat
a))
        episode_stats['episode_returns'].append(total_reward)
        episode_stats['states_visited'].append([step[0] for step
in episode_data])

        # Step 5: G ← 0
        G = 0

        # Step 6: For t = T-1, T-2, ..., 0:
        for t in range(len(episode_data) - 1, -1, -1):
            state_t, action_t, reward_t_plus_1 = episode_data[t]

            # Step 7: G ← γG + R_{t+1}
            G = gamma * G + reward_t_plus_1

            # Step 8: If S_t ∉ {S₀, S₁, ..., S_{t-1}}: (First-vis
it condition)
            states_before_t = [episode_data[i][0] for i in range
(t)]
            if (state_t not in states_before_t) or (not first_vis
it):

                # Step 9: Append G to Returns(S_t)
                returns[state_t].append(G)

                # Step 10: V(S_t) ← average(Returns(S_t))
                value_function[state_t] = np.mean(returns[state_
t])

        # Show progress with animation
        if show_animation and (episode % plot_interval == 0 or ep
isode == num_episodes - 1):
            update_mc_value_function_plot(value_function,
                                          "Monte Carlo Value Functi
on Learning",
```

```
                                        episode=episode+1)
            if episode < num_episodes - 1:
                time.sleep(0.5)  # Brief pause for animation

    # Print final results
    print(f"\nCompleted {num_episodes} episodes with γ = {gamm
a}")
    print("Estimated Value Function V^π(s):")
    for state in range(env.observation_space.n):
        visits = len(returns[state])
        print(f"State {state}: V = {value_function[state]:.4f} (v
isits: {visits})")

    print(f"\nEnvironment Description:")
    if env_name == 'FrozenLake-v1':
        print("FrozenLake 4x4 grid (deterministic)")
        print("Goal: Reach the frisbee (state 15) from start (sta
te 0)")
        print("Holes at states: 5, 7, 11, 12")

    # Calculate additional statistics
    avg_episode_length = np.mean(episode_stats['episode_length
s'])
    success_rate = np.mean([r > 0 for r in episode_stats['episode
_returns']])

    print(f"\nStatistics:")
    print(f"Average episode length: {avg_episode_length:.1f}")
    print(f"Success rate: {success_rate:.3f}")

    return value_function, returns, episode_stats
```

In [2]:

```
# Run First-Visit Monte Carlo Prediction
# Call the function with different parameter configurations

print("🎬 Running First-Visit Monte Carlo Prediction")
print("=" * 50)

# Example 1: Standard run with animation
print("\n📊 Example 1: Standard Monte Carlo with Animation")
value_function, returns, stats = monte_carlo_prediction(
    num_episodes=1000,
    gamma=0.95,
    initial_value='zeros',
    show_animation=True,
    plot_interval=100,
    first_visit=True
)

# Display some key results
print(f"\n🎯 Key Results:")
print(f"States with highest values: {np.argsort(value_function)[-
3:]}")
print(f"Value of goal state (15): {value_function[15]:.4f}")
print(f"Value of start state (0): {value_function[0]:.4f}")
```


Monte Carlo Value Function Learning (Episode 1000)
S
0.010
s0
0.008
s1
0.018
s2
0.020
s3
0.011
s4
H
s5
0.045
s6
H
s7
0.030
s8
0.077
s9
0.140
s10
H
s11
H
s12
0.162
s13
0.493
s14
0.000
s15
G
FrozenLake 4x4: S=Start(0), F=Frozen, H=Hole, G=Goal(15)
Values range: [0.000, 0.493]

```
Completed 1000 episodes with γ = 0.95
Estimated Value Function V^π(s):
State 0: V = 0.0102 (visits: 1000)
State 1: V = 0.0077 (visits: 572)
State 2: V = 0.0175 (visits: 240)
State 3: V = 0.0203 (visits: 104)
State 4: V = 0.0112 (visits: 655)
State 5: V = 0.0000 (visits: 0)
State 6: V = 0.0448 (visits: 129)
State 7: V = 0.0000 (visits: 0)
State 8: V = 0.0297 (visits: 284)
State 9: V = 0.0774 (visits: 127)
State 10: V = 0.1397 (visits: 78)
State 11: V = 0.0000 (visits: 0)
State 12: V = 0.0000 (visits: 0)
State 13: V = 0.1618 (visits: 37)
State 14: V = 0.4928 (visits: 34)
State 15: V = 0.0000 (visits: 0)

Environment Description:
FrozenLake 4x4 grid (deterministic)
Goal: Reach the frisbee (state 15) from start (state 0)
Holes at states: 5, 7, 11, 12

Statistics:
Average episode length: 7.6
Success rate: 0.017

🎯 Key Results:
States with highest values: [10 13 14]
Value of goal state (15): 0.0000
Value of start state (0): 0.0102
```

Every-visit MC method

The every-visit Monte Carlo method estimates the value of a state as the average of the returns following all visits to that state, not just the first visit in each episode.

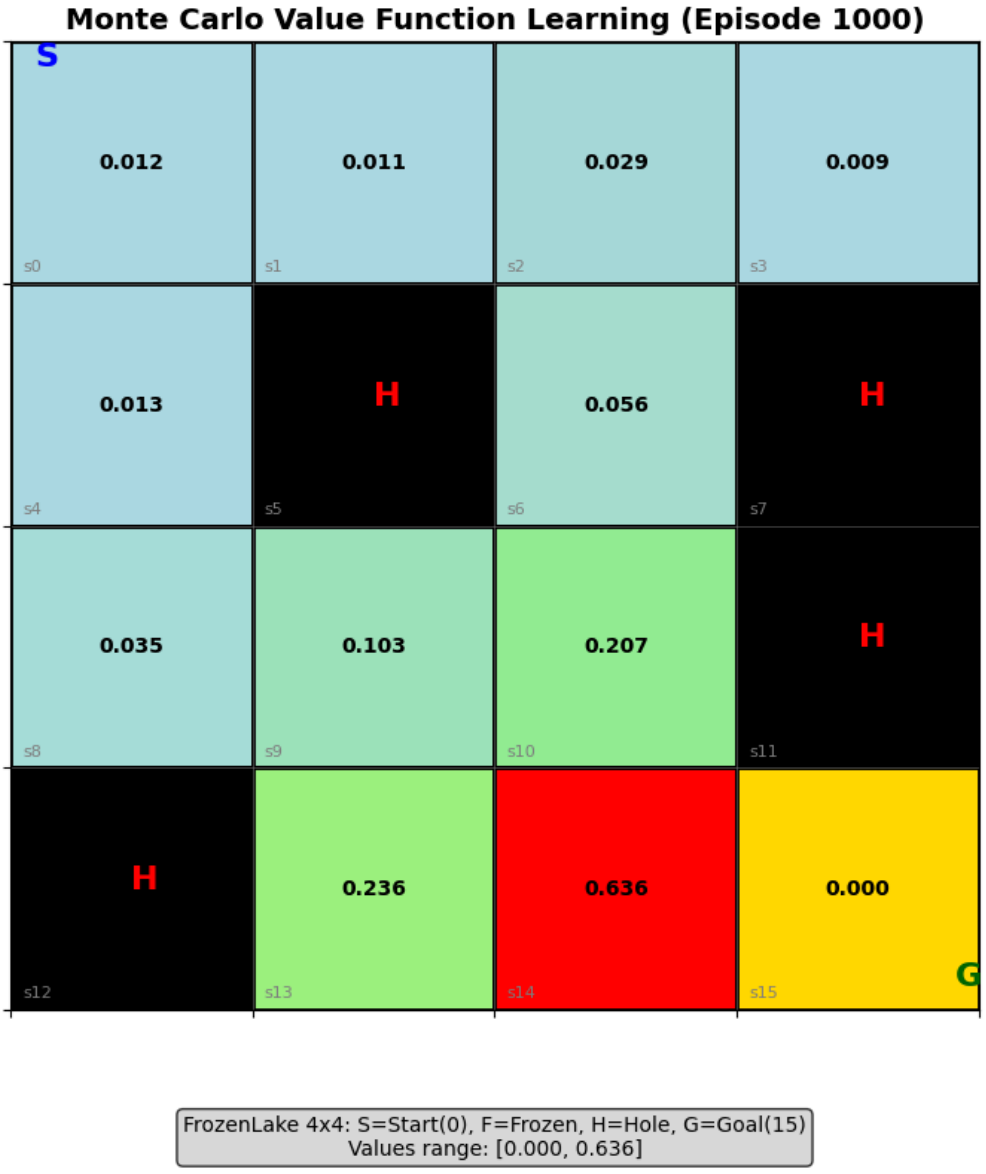
... as the algorithm before, just without this condition:

$$\begin{aligned}
&\textbf{8. If } s_t \notin \{s_0, s_1, \ldots, s_{t-1}\} : (\backslash\text{texttt } \backslash\text{tiny This is the first-visit condition!}) \\
&\quad \textbf{9. } \text{Append } G \text{ to } Returns(s_t) \\
&\quad \textbf{10. } V(S_t) \leftarrow \text{average}(Returns(s_t))
\end{aligned}$$

- Estimates value from all visits
- Average of returns
- Not limited to first visit

In [3]:

```
# Run First-Visit Monte Carlo Prediction
# Call the function with different parameter configurations

print("🎬 Running Last-Visit Monte Carlo Prediction")
print("=" * 50)

# Example 1: Standard run with animation
print("\n📊 Example 1: Standard Monte Carlo with Animation")
value_function, returns, stats = monte_carlo_prediction(
    num_episodes=1000,
    gamma=0.95,
    initial_value='zeros',
    show_animation=True,
    plot_interval=100,
    first_visit=False
)

# Display some key results
print(f"\n🎯 Key Results:")
print(f"States with highest values: {np.argsort(value_function)[-
3:]}")
print(f"Value of goal state (15): {value_function[15]:.4f}")
print(f"Value of start state (0): {value_function[0]:.4f}")
```


Monte Carlo Value Function Learning (Episode 1000)
S
0.012
s0
0.011
s1
0.029
s2
0.009
s3
0.013
s4
H
s5
0.056
s6
H
s7
0.035
s8
0.103
s9
0.207
s10
H
s11
H
s12
0.236
s13
0.636
s14
0.000
s15
G
FrozenLake 4x4: S=Start(0), F=Frozen, H=Hole, G=Goal(15)
Values range: [0.000, 0.636]

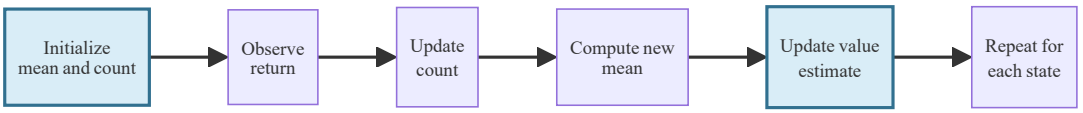
```
Completed 1000 episodes with γ = 0.95
Estimated Value Function V^π(s):
State 0: V = 0.0121 (visits: 3250)
State 1: V = 0.0105 (visits: 1254)
State 2: V = 0.0293 (visits: 607)
State 3: V = 0.0086 (visits: 280)
State 4: V = 0.0134 (visits: 1273)
State 5: V = 0.0000 (visits: 0)
State 6: V = 0.0562 (visits: 184)
State 7: V = 0.0000 (visits: 0)
State 8: V = 0.0354 (visits: 451)
State 9: V = 0.1033 (visits: 144)
State 10: V = 0.2073 (visits: 91)
State 11: V = 0.0000 (visits: 0)
State 12: V = 0.0000 (visits: 0)
State 13: V = 0.2363 (visits: 61)
State 14: V = 0.6355 (visits: 52)
State 15: V = 0.0000 (visits: 0)

Environment Description:
FrozenLake 4x4 grid (deterministic)
Goal: Reach the frisbee (state 15) from start (state 0)
Holes at states: 5, 7, 11, 12

Statistics:
Average episode length: 7.6
Success rate: 0.021

🎯 Key Results:
States with highest values: [10 13 14]
Value of goal state (15): 0.0000
Value of start state (0): 0.0121
```

Incremental implementation of MC prediction

Motivation: The previous Monte Carlo implementation has significant limitations:

- **Memory inefficiency**: Stores all returns for every state across all episodes
- **Computational inefficiency**: Recalculates the entire average from scratch after each episode
- **Scalability issues**: Memory requirements grow linearly with the number of episodes
- **Delayed updates**: Value estimates are only updated after collecting many returns

The incremental approach addresses these shortcomings by updating value estimates immediately after each episode using only the new return, without storing historical data.



Incremental averaging for MC

- Maintains mean and count for each state
- Updates value estimate with each new return
- Reduces memory usage

Incremental First-Visit Monte Carlo Prediction Algorithm

Incremental First-Visit Monte Carlo Prediction
Input: π (policy to evaluate), γ (discount factor)
Output: V^π (value function approximation)

Algorithm:
1. Initialize $V(s) \leftarrow 0$ for all $s \in \mathcal{S}$
2. Initialize $N(s) \leftarrow 0$ for all $s \in \mathcal{S}$

3. Repeat forever (for each episode):
4. Generate episode: $s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, r_T$
5. $G \leftarrow 0$

6. For $t = T-1, T-2, \ldots, 0$:
7. $G \leftarrow \gamma G + r_{t+1}$
8. If $s_t \notin \{s_0, s_1, \ldots, s_{t-1}\}$ (first-visit condition):
9. $N(s_t) \leftarrow N(s_t) + 1$
10. $V(s_t) \leftarrow V(s_t) + \frac{1}{N(s_t)}[G - V(s_t)]$

Key Simplifications from Standard Monte Carlo:

- **Line 2**: Track visit count $N(s_t)$ instead of storing all returns
- **Line 9**: Increment visit counter for each first visit
- **Line 10**: **Sample averaging**: Uses $\alpha = \frac{1}{N(s_t)}$ for true Monte Carlo estimation

Memory Complexity:

- **Standard MC**: $O(\text{total visits})$ - stores all returns in lists
- **Incremental MC**: $O(|\mathcal{S}|)$ - only current estimates and visit counts

Mathematical Equivalence: The incremental update with $\alpha = \frac{1}{N(s)}$ is mathematically identical to computing the sample average:

$$V_n(s) = \frac{1}{n}\sum_{i=1}^{n} G_i(s) = V_{n-1}(s) + \frac{1}{n}[G_n(s) - V_{n-1}(s)]$$

where $V_n(s)$ is the estimate after n visits to state s, and $G_n(s)$ is the n-th return from state s.

In [4]:
```
def monte_carlo_prediction_incremental(env_name='FrozenLake-v1',
                                       num_episodes=1000,
                                       gamma=0.95,
                                       initial_value='zeros',
                                       plot_interval=100,
                                       show_animation=True,
                                       max_episode_steps=50,
                                       is_slippery=False,
                                       policy_fn=None,
                                       first_visit=True):
    """
    Incremental First-Visit Monte Carlo Prediction Algorithm

    Uses sample averaging (α = 1/N(s)) for true Monte Carlo estim
ation:
    1. Initialize V(s) arbitrarily for all s ∈ S
    2. Initialize N(s) ← 0 for all s ∈ S (visit counts)
    3. Repeat forever (or for num_episodes)
    4. Generate episode following π
    5. G ← 0
    6. For t = T-1, T-2, ..., 0:
    7. G ← γG + R_{t+1}
    8. If S_t ∉ {S₀, S₁, ..., S_{t-1}}: (First-visit condition)
    9. N(S_t) ← N(S_t) + 1
    10. V(S_t) ← V(S_t) + (1/N(S_t)) * [G - V(S_t)]  # Sample ave
raging

    Args:
        env_name: Gymnasium environment name
        num_episodes: Number of episodes to run
        gamma: Discount factor
        initial_value: 'zeros', 'random', or array of initial val
ues
        plot_interval: Show plot every N episodes (if show_animat
ion=True)
        show_animation: Whether to show animated plots
        max_episode_steps: Maximum steps per episode
        is_slippery: Whether environment has stochastic transitio
ns
        policy_fn: Policy function (if None, uses random policy)
        first_visit: Whether to use first-visit or every-visit MC

    Returns:
        tuple: (value_function, visit_counts, episode_stats)
            - value_function: Final estimated value function
            - visit_counts: Number of visits to each state
            - episode_stats: Dictionary with statistics
    """
    import numpy as np
    import gymnasium as gym

    # Create environments
    if show_animation:
        rendered_env = gym.make(env_name, is_slippery=is_slipper
y, render_mode="human",
                                max_episode_steps=max_episode_step
```

```
s, reward_schedule=(1, 0, 0))
    hidden_env = gym.make(env_name, is_slippery=is_slippery, rend
er_mode="ansi",
                          max_episode_steps=max_episode_steps, rew
ard_schedule=(1, 0, 0))

    # Use hidden environment as default
    env = hidden_env

    # Step 1: Initialize V(s) arbitrarily for all s ∈ S
    if initial_value == 'zeros':
        value_function = np.zeros(env.observation_space.n)
    elif initial_value == 'random':
        value_function = np.random.rand(env.observation_space.n)
    else:
        value_function = np.array(initial_value)

    # Step 2: Initialize N(s) ← 0 for all s ∈ S (visit counts)
    visit_counts = np.zeros(env.observation_space.n)

    # Define default policy π to evaluate
    if policy_fn is None:
        def policy_fn(state):
            """Simple random policy for demonstration"""
            return np.random.choice(env.action_space.n)

    # Statistics tracking
    episode_stats = {
        'episode_lengths': [],
        'episode_returns': [],
        'states_visited': [],
        'value_updates': []  # Track how many updates per episode
    }

    print("Incremental First-Visit Monte Carlo Prediction")
    print("=" * 50)
    print(f"Environment: {env_name}")
    print(f"Episodes: {num_episodes}, γ = {gamma}")
    print(f"Update rule: Sample averaging (α = 1/N(s))")
    print(f"Initial values: {initial_value}")

    # Show initial value function
    if show_animation:
        update_mc_value_function_plot(value_function, "Initial Va
lue Function (Incremental)", episode=0)
        time.sleep(1)

    # Step 3: Repeat forever (or for num_episodes)
    for episode in range(num_episodes):
        # Use rendered environment occasionally for visualization
        if show_animation and episode % plot_interval == 0:
            print(f"Episode {episode}/{num_episodes}")
            current_env = rendered_env if show_animation else hid
den_env
        else:
            current_env = hidden_env
```

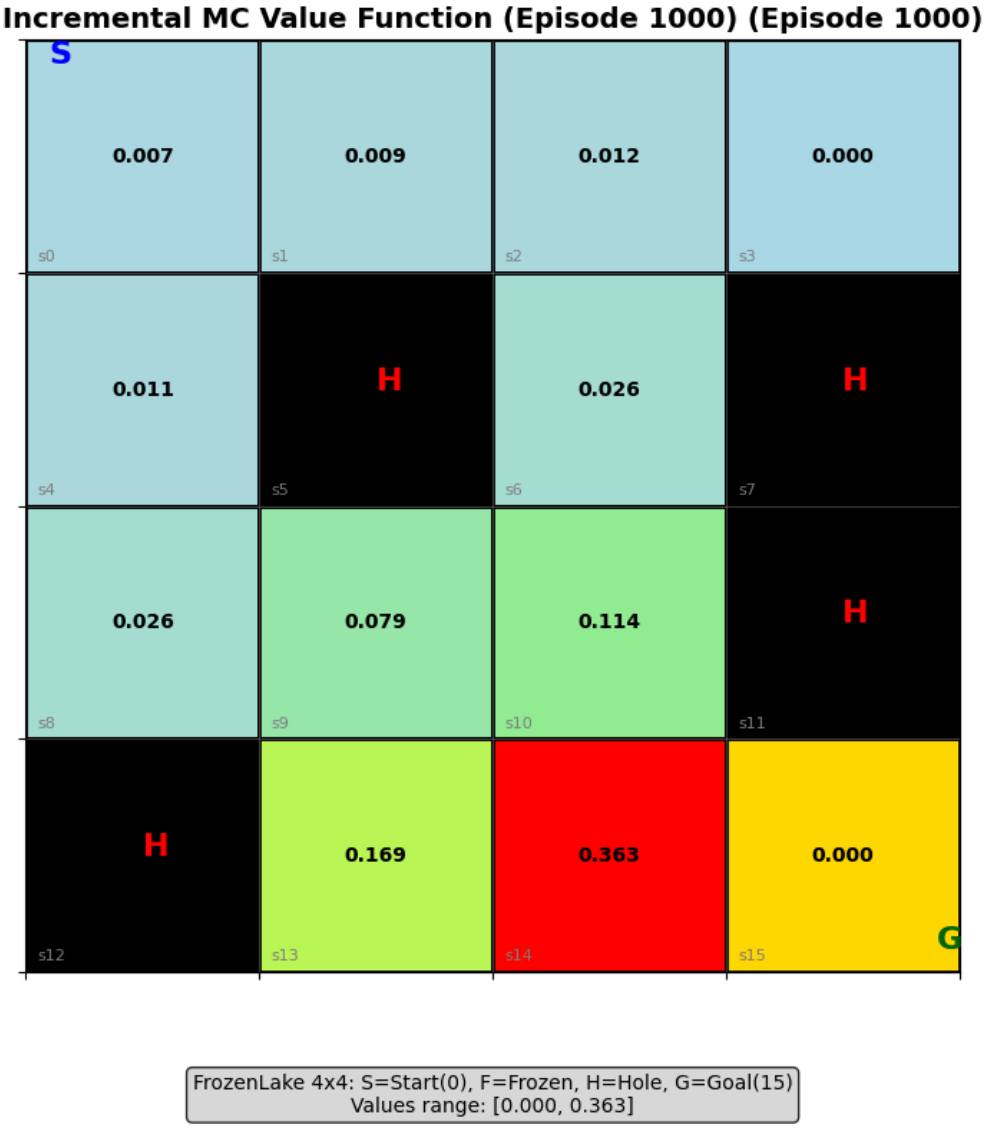
```
        # Step 4: Generate episode following π: S₀, A₀, R₁, S₁, A
₁, R₂, ..., S_{T-1}, A_{T-1}, R_T
        episode_data = []
        state = current_env.reset()[0]
        done = False
        total_reward = 0

        # Generate complete episode
        while not done:
            action = policy_fn(state)
            next_state, reward, done, _, _ = current_env.step(act
ion)
            episode_data.append((state, action, reward))  # Store
(S_t, A_t, R_{t+1})
            total_reward += reward
            state = next_state

        # Track episode statistics
        episode_stats['episode_lengths'].append(len(episode_dat
a))
        episode_stats['episode_returns'].append(total_reward)
        episode_stats['states_visited'].append([step[0] for step
in episode_data])

        # Step 5: G ← 0
        G = 0
        updates_this_episode = 0

        # Step 6: For t = T-1, T-2, ..., 0:
        for t in range(len(episode_data) - 1, -1, -1):
            state_t, action_t, reward_t_plus_1 = episode_data[t]

            # Step 7: G ← γG + R_{t+1}
            G = gamma * G + reward_t_plus_1

            # Step 8: If S_t ∉ {S₀, S₁, ..., S_{t-1}}: (First-vis
it condition)
            states_before_t = [episode_data[i][0] for i in range
(t)]
            if (state_t not in states_before_t) or (not first_vis
it):

                # Step 9: N(S_t) ← N(S_t) + 1
                visit_counts[state_t] += 1

                # Step 10: V(S_t) ← V(S_t) + (1/N(S_t)) * [G - V
(S_t)]
                # Sample averaging: α = 1/N(S_t)
                step_size = 1.0 / visit_counts[state_t]

                # Incremental update formula
                old_value = value_function[state_t]
                value_function[state_t] = old_value + step_size *
(G - old_value)
                updates_this_episode += 1

        # Track updates per episode
```

```
        episode_stats['value_updates'].append(updates_this_episod
e)

        # Show progress with animation
        if show_animation and (episode % plot_interval == 0 or ep
isode == num_episodes - 1):
            update_mc_value_function_plot(value_function,
                                          f"Incremental MC Value Fu
nction (Episode {episode+1})",
                                          episode=episode+1)
            if episode < num_episodes - 1:
                time.sleep(0.5)  # Brief pause for animation

    # Print final results
    print(f"\nCompleted {num_episodes} episodes with γ = {gamm
a}")
    print("Estimated Value Function V^π(s) (Incremental Update
s):")
    for state in range(env.observation_space.n):
        visits = int(visit_counts[state])
        print(f"State {state}: V = {value_function[state]:.4f} (v
isits: {visits})")

    print(f"\nEnvironment Description:")
    if env_name == 'FrozenLake-v1':
        print("FrozenLake 4x4 grid (deterministic)")
        print("Goal: Reach the frisbee (state 15) from start (sta
te 0)")
        print("Holes at states: 5, 7, 11, 12")

    # Calculate additional statistics
    avg_episode_length = np.mean(episode_stats['episode_length
s'])
    success_rate = np.mean([r > 0 for r in episode_stats['episode
_returns']])
    total_updates = np.sum(episode_stats['value_updates'])
    avg_updates_per_episode = np.mean(episode_stats['value_update
s'])

    print(f"\nIncremental Update Statistics:")
    print(f"Total value updates: {total_updates}")
    print(f"Average updates per episode: {avg_updates_per_episod
e:.1f}")
    print(f"Average episode length: {avg_episode_length:.1f}")
    print(f"Success rate: {success_rate:.3f}")

    # Memory efficiency note
    total_visits = np.sum(visit_counts)
    print(f"\nMemory Efficiency:")
    print(f"Total state visits: {int(total_visits)}")
    print(f"Memory used: O(|S|) = O({env.observation_space.n}) fo
r value function + visit counts")
    print(f"vs. Standard MC: O(Total Visits) = O({int(total_visit
s)}) for storing all returns")

    return value_function, visit_counts, episode_stats
```

In [5]:

```
# Run Incremental First-Visit Monte Carlo Prediction
# Uses sample averaging (alpha = 1/N(s)) for true Monte Carlo est
imation

print("Running Incremental First-Visit Monte Carlo Prediction")
print("=" * 60)

print("\nIncremental Monte Carlo with Sample Averaging")
print("Using alpha(s) = 1/N(s) for each state (true Monte Carlo a
pproach)")

value_function, visit_counts, stats = monte_carlo_prediction_incr
emental(
    num_episodes=1000,
    gamma=0.95,
    initial_value='zeros',
    show_animation=True,
    plot_interval=100,
    first_visit=True
)

print(f"\nFinal Results:")
print(f"States with highest values: {np.argsort(value_function)[-
3:]}")
print(f"Value of goal state (15): {value_function[15]:.4f}")
print(f"Value of start state (0): {value_function[0]:.4f}")

# Display key characteristics of incremental approach
print(f"\nIncremental Monte Carlo Characteristics:")
print(f"Total value updates: {np.sum(stats['value_updates'])}")
print(f"Average updates per episode: {np.mean(stats['value_update
s']):.1f}")
print(f"Success rate: {np.mean([r > 0 for r in stats['episode_ret
urns']]):.3f}")
print(f"Average episode length: {np.mean(stats['episode_length
s']):.1f}")

# Show memory efficiency benefit
total_visits = np.sum(visit_counts)
print(f"\nMemory Efficiency Demonstration:")
print(f"Total state visits: {int(total_visits)}")
print(f"Memory used by incremental MC: O(|S|) = {len(value_functi
on)} values + {len(visit_counts)} counters")
print(f"Memory saved vs standard MC: {int(total_visits)} - {len(v
alue_function)} = {int(total_visits - len(value_function))} retur
n values")

# Show the learning curve in terms of visit distribution
print(f"\nVisit Distribution:")
print(f"Most visited states: {np.argsort(visit_counts)[-5:]}")
print(f"Least visited states: {np.argsort(visit_counts)[:5]}")
for state in range(16):
    if visit_counts[state] > 0:
        print(f"State {state}: {int(visit_counts[state]):3d} visi
ts -> V = {value_function[state]:.4f}")
    else:
```

```
        print(f"State {state}:   0 visits -> V = {value_function
[state]:.4f} (unvisited)")

print("\nIncremental Monte Carlo completed successfully!")
print("Key advantage: Memory usage independent of number of episo
des/visits")
```


Incremental MC Value Function (Episode 1000) (Episode 1000)
S
0.007
0.009
0.012
0.000
s0
s1
s2
s3
0.011
H
0.026
H
s4
s5
s6
s7
0.026
0.079
0.114
H
s8
s9
s10
s11
H
0.169
0.363
0.000
G
s12
s13
s14
s15
FrozenLake 4x4: S=Start(0), F=Frozen, H=Hole, G=Goal(15)
Values range: [0.000, 0.363]

```
Completed 1000 episodes with γ = 0.95
Estimated Value Function V^π(s) (Incremental Updates):
State 0: V = 0.0075 (visits: 1000)
State 1: V = 0.0087 (visits: 636)
State 2: V = 0.0125 (visits: 290)
State 3: V = 0.0000 (visits: 130)
State 4: V = 0.0108 (visits: 592)
State 5: V = 0.0000 (visits: 0)
State 6: V = 0.0264 (visits: 144)
State 7: V = 0.0000 (visits: 0)
State 8: V = 0.0258 (visits: 262)
State 9: V = 0.0793 (visits: 120)
State 10: V = 0.1143 (visits: 80)
State 11: V = 0.0000 (visits: 0)
State 12: V = 0.0000 (visits: 0)
State 13: V = 0.1695 (visits: 48)
State 14: V = 0.3630 (visits: 36)
State 15: V = 0.0000 (visits: 0)

Environment Description:
FrozenLake 4x4 grid (deterministic)
Goal: Reach the frisbee (state 15) from start (state 0)
Holes at states: 5, 7, 11, 12

Incremental Update Statistics:
Total value updates: 3338
Average updates per episode: 3.3
Average episode length: 7.6
Success rate: 0.014

Memory Efficiency:
Total state visits: 3338
Memory used: O(|S|) = O(16) for value function + visit counts
vs. Standard MC: O(Total Visits) = O(3338) for storing all return
s

Final Results:
States with highest values: [10 13 14]
Value of goal state (15): 0.0000
Value of start state (0): 0.0075

Incremental Monte Carlo Characteristics:
Total value updates: 3338
Average updates per episode: 3.3
Success rate: 0.014
Average episode length: 7.6

Memory Efficiency Demonstration:
Total state visits: 3338
Memory used by incremental MC: O(|S|) = 16 values + 16 counters
Memory saved vs standard MC: 3338 - 16 = 3322 return values

Visit Distribution:
Most visited states: [8 2 4 1 0]
Least visited states: [ 7  5 12 11 15]
State 0: 1000 visits -> V = 0.0075
```

```
State 1: 636 visits -> V = 0.0087
State 2: 290 visits -> V = 0.0125
State 3: 130 visits -> V = 0.0000
State 4: 592 visits -> V = 0.0108
State 5:   0 visits -> V = 0.0000 (unvisited)
State 6: 144 visits -> V = 0.0264
State 7:   0 visits -> V = 0.0000 (unvisited)
State 8: 262 visits -> V = 0.0258
State 9: 120 visits -> V = 0.0793
State 10:  80 visits -> V = 0.1143
State 11:   0 visits -> V = 0.0000 (unvisited)
State 12:   0 visits -> V = 0.0000 (unvisited)
State 13:  48 visits -> V = 0.1695
State 14:  36 visits -> V = 0.3630
State 15:   0 visits -> V = 0.0000 (unvisited)

Incremental Monte Carlo completed successfully!
Key advantage: Memory usage independent of number of episodes/vis
its
```

Monte Carlo On-Policy Control

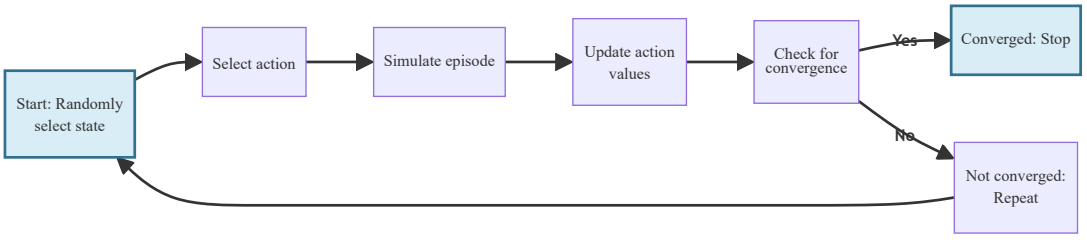
Motivation: Beyond just evaluating a policy, we want to find the optimal policy without a model. On-policy control alternates between evaluating the current policy and improving it, all while generating data from the same policy being optimized.

What it does: Discovers an optimal (or near-optimal) policy by iteratively improving the policy used to generate experience data.

How it works:

1. **Generalized Policy Iteration (GPI)**: Alternates between two steps, as in Dynamic Programming:
   - **Policy Evaluation**: Estimate $Q^\pi(s, a)$ using Monte Carlo prediction with episodes from current policy
   - **Policy Improvement**: Make policy greedy with respect to current Q-values: $\pi(s) = \arg\max_a Q(s, a)$
2. **Exploration requirement**: Must ensure all state-action pairs are visited, using methods like:
   - **Exploring Starts**: Begin episodes with random state-action pairs
   - **Epsilon-greedy policies**: Take random actions with probability ϵ to maintain exploration
3. **Convergence**: Process converges toward optimal policy π^* and optimal Q-function Q^*

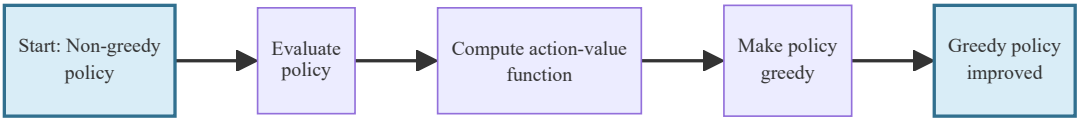
Exploration with Exploring Starts (ES)



MC ES algorithm description

- Randomly selects starting state and action
- Ensures all state-action pairs are visited
- Unbiased estimation of action values
- Converges to optimal policy and action-value function

Policy improvement with MC methods



Greedy policy improvement

- Monte Carlo control
- Policy becomes greedy
- Selects action with highest value (as in value iteration in DP)

Policy improvement theorem

- Policy improvement theorem
- Guarantees monotonic improvement
- New policy $\geq$ previous policy

Monte Carlo ES (Exploring Starts) Control

Monte Carlo ES (Exploring Starts) Control
Input: Environment with states $\mathcal{S}$ and actions $\mathcal{A}$
Output: π^* (optimal policy), Q^* (optimal action-value function)

Algorithm:

1. Initialize:
   $Q(s,a) \leftarrow$ arbitrary for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
   $\pi(s) \leftarrow$ arbitrary for all $s \in \mathcal{S}$
   $\text{Returns}(s,a) \leftarrow$ empty list for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

2. Repeat forever (for each episode):

3. Choose random $s_0 \in \mathcal{S}, a_0 \in \mathcal{A}(s_0)$ (exploring starts)
4. Generate episode starting from (s_0, a_0) following π :
   $s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, r_T$

5. $G \leftarrow 0$
6. For $t = T-1, T-2, \ldots, 0$:
7. $G \leftarrow \gamma G + r_{t+1}$
8. If $(s_t, a_t) \notin \{(s_0,a_0),(s_1,a_1),\ldots,(s_{t-1},a_{t-1})\}$: (first-visit)
9. Append G to $\text{Returns}(s_t, a_t)$
10. $Q(s_t, a_t) \leftarrow \text{average}(\text{Returns}(s_t, a_t))$
11. $\pi(s_t) \leftarrow \arg\max_a Q(s_t, a)$ (policy improvement)

Key Components:

- **Line 3**: **Exploring Starts** - Randomly initialize each episode with any state-action pair (s_0, a_0)
- **Line 4**: Follow current policy π after the initial random state-action pair
- **Lines 5-10**: **Policy Evaluation** - Estimate Q^π using first-visit MC averaging
- **Line 11**: **Policy Improvement** - Make policy greedy w.r.t. current Q-values

Convergence: Under exploring starts assumption, this algorithm converges to the optimal policy π^* and optimal action-value function Q^*.

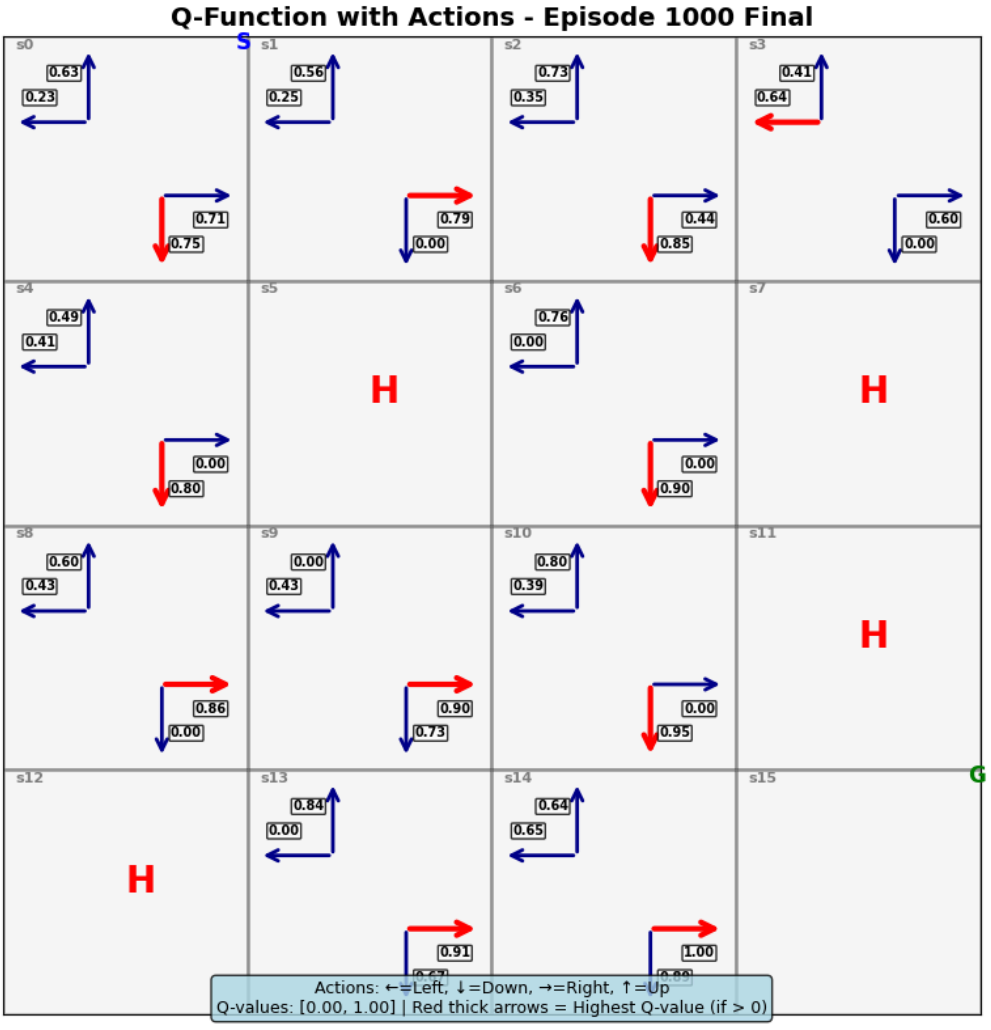
In [6]:

```
# Reimport helpers (for development)
import importlib
import helper_functions
importlib.reload(helper_functions)
from helper_functions import *


def monte_carlo_es_control(env_name='FrozenLake-v1', num_episodes
=5000, gamma=0.95,
                           plot_interval=500, animate=True, explor
ing_starts=True):
    """
    Monte Carlo ES Control Algorithm (compact version)
    Follows pseudocode: Initialize Q, π, Returns → Loop episodes
with exploring starts

    Args:
        exploring_starts: If True, use random initial states. If
False, always start from s_0.
    """
    import gymnasium as gym
    from collections import defaultdict

    max_ep_steps = 50
    headless_env = gym.make(env_name, is_slippery=False, render_m
ode="ansi",
                    max_episode_steps=max_ep_steps, reward_schedul
e=(1, 0, 0))
    render_env = gym.make(env_name, is_slippery=False, render_mod
e="human",
                    max_episode_steps=max_ep_steps, reward_schedul
e=(1, 0, 0))
    n_states, n_actions = render_env.observation_space.n, render_
env.action_space.n

    # Step 1: Initialize Q(s,a), π(s), Returns(s,a)
    Q = np.zeros((n_states, n_actions))
    policy = np.random.randint(0, n_actions, size=n_states)
    returns = defaultdict(list)

    es_status = "WITH" if exploring_starts else "WITHOUT"
    print(f"MC Control {es_status} Exploring Starts: {n_states} s
tates, {n_actions} actions, {num_episodes} episodes, γ={gamma}")

    if animate:
        update_mc_control_plot(Q, 0, "Initial")
        time.sleep(2)

    # Step 2: Repeat for each episode
    for episode in range(1, num_episodes + 1):
        if episode % plot_interval == 0 and animate:
            env = render_env
        else:
            env = headless_env

        # Step 3: Determine starting state and action
```

```
        if exploring_starts:
            # Exploring starts - random s_0 and a_0
            terminal_states = [5, 7, 11, 12, 15]
            valid_starts = [s for s in range(n_states) if s not i
n terminal_states]
            s_0 = np.random.choice(valid_starts)
            a_0 = np.random.randint(0, n_actions)  # Random initi
al action
            env.reset()
            env.unwrapped.s = s_0  # Force start at random state
        else:
            # Always start from default state with policy action
            s_0, _ = env.reset()
            a_0 = policy[s_0]  # Use policy for initial action

        # Step 4: Generate episode starting from (s_0, a_0)
        episode_data = []
        state, done = s_0, False
        action = a_0  # Use initial action (random if ES, policy
otherwise)

        step_count = 0
        while not done and step_count < max_ep_steps:
            next_state, reward, done, _, _ = env.step(action)
            episode_data.append((int(state), int(action), float(r
eward)))
            state = next_state
            action = policy[state] if not done else 0  # Follow p
olicy after first action
            step_count += 1

        # Steps 5-11: Update Q and π using first-visit MC
        G = 0
        for t in range(len(episode_data) - 1, -1, -1):
            state_t, action_t, reward_t_plus_1 = episode_data[t]
            G = gamma * G + reward_t_plus_1

            # First-visit check
            sa_pairs_before = [(episode_data[i][0], episode_data
[i][1]) for i in range(t)]
            if (state_t, action_t) not in sa_pairs_before:
                returns[(state_t, action_t)].append(G)
                Q[state_t, action_t] = np.mean(returns[(state_t,
action_t)])
                policy[state_t] = np.argmax(Q[state_t, :])

        # Animate
        if animate and episode % plot_interval == 0:
            update_mc_control_plot(Q, episode)
            time.sleep(2)
            max_v = np.max(Q, axis=1).max()
            print(f"Episode {episode:4d}: Max Q(s,a) = {max_v:.3
f}, SA-pairs explored = {len(returns)}")

    if animate:
        update_mc_control_plot(Q, num_episodes, "Final")
        plt.ioff()
```

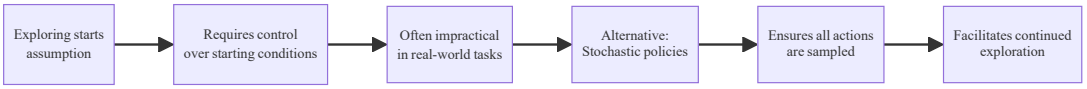
```
        # Keep the final plot open by not clearing output

    print(f"\nConverged! SA-pairs explored: {len(returns)}/{n_sta
tes*n_actions}, Max Q(s,a): {np.max(Q, axis=1).max():.3f}")

    env.close()
    return Q, policy, returns
```

In [7]:
```
# Run MC Control with Exploring Starts
Q_optimal, pi_optimal, returns_dict = monte_carlo_es_control(
    env_name='FrozenLake-v1',
    num_episodes=1000,
    gamma=0.95,
    plot_interval=100,
    animate=True,
    exploring_starts=True  # Set to False to disable exploring st
arts
)
```




```
Converged! SA-pairs explored: 44/64, Max Q(s,a): 1.000
```

Limitations of exploring starts assumption



Unrealistic in real-world tasks

- Exploring starts assumption
- Requires control over starting conditions
- Often impractical in real-world tasks

Alternative: stochastic policies

- Stochastic policies (e.g., ε-greedy)
- Ensures all actions are sampled
- Facilitates continued exploration

Monte Carlo On-Policy Control with ε-Greedy

Motivation: Exploring starts is impractical in real applications. We need a method that ensures sufficient exploration while learning the optimal policy. ε-greedy policies provide a practical solution by maintaining exploration during learning.

What it does: Learns an optimal policy by alternating between policy evaluation and improvement, using ε-greedy policies to ensure continued exploration without requiring exploring starts.

Key insight: Instead of requiring exploring starts, we use ε-greedy policies where:

- With probability $1 - \varepsilon$: Choose the greedy action (exploitation)
- With probability ε: Choose a random action (exploration)

Exploration with ε-greedy policies

ε-Greedy Policy Selection
Input: $Q(s, a)$ (action-value function), s (current state), ϵ (exploration rate)
Output: a (selected action)

Algorithm:
1. Generate random number: $p \sim \text{Uniform}(0, 1)$

2. If $p < \epsilon$:
  3. $a \leftarrow$ random action from $\mathcal{A}(s)$ (explore)

4. Else:
  5. $a \leftarrow \arg\max_{a'} Q(s, a')$ (exploit)

6. Return a

Key Properties:

- **Exploration probability**: With probability ϵ, select random action (line 3)
- **Exploitation probability**: With probability $1 - \epsilon$, select greedy action (line 5)
- **Coverage guarantee**: Every action has probability $\geq \frac{\epsilon}{|\mathcal{A}(s)|}$ of being selected
- **ε-soft policy**: $\pi(a|s) > 0$ for all $a \in \mathcal{A}(s)$, ensuring continual exploration

Mathematical Formulation:

ε-soft policy: $\pi(a|s) \geq \frac{\epsilon}{|\mathcal{A}(s)|}$ for all $a \in \mathcal{A}(s)$

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|} & \text{if } a = \arg\max_{a'} Q(s, a') \\ \frac{\epsilon}{|\mathcal{A}(s)|} & \text{otherwise} \end{cases}$$

Monte Carlo On-Policy Control with ε-Greedy

Monte Carlo On-Policy Control with ε-Greedy
Input: Environment with states $\mathcal{S}$ and actions $\mathcal{A}, \epsilon > 0$ (exploration rate)
Output: π_ϵ (ε-greedy policy), Q (action-value function)

Algorithm:

1. Initialize:
   $Q(s, a) \leftarrow$ arbitrary for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
   $\pi \leftarrow \varepsilon$-greedy policy derived from Q
   $\text{Returns}(s, a) \leftarrow$ empty list for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

2. Repeat forever (for each episode):

3. Generate episode using π :
   $s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, r_T$

4. $G \leftarrow 0$
5. For $t = T-1, T-2, \ldots, 0$:
6. $G \leftarrow \gamma G + r_{t+1}$
7. If $(s_t, a_t) \notin \{(s_0, a_0), (s_1, a_1), \ldots, (s_{t-1}, a_{t-1})\}$: (first-visit)
8. Append G to $\text{Returns}(s_t, a_t)$
9. $Q(s_t, a_t) \leftarrow \text{average}(\text{Returns}(s_t, a_t))$
10. $\pi \leftarrow \varepsilon$-greedy policy w.r.t. Q

Key Components:

- **Line 2**: Initialize policy as ε-greedy (not fully greedy like MC ES)
- **Line 3**: Generate episodes using current ε-greedy policy (no exploring starts needed)
- **Lines 4-9**: **Policy Evaluation** - Estimate Q^{π_ϵ} using first-visit MC averaging
- **Line 10**: **Policy Improvement** - Update policy to be ε-greedy w.r.t. current Q-values

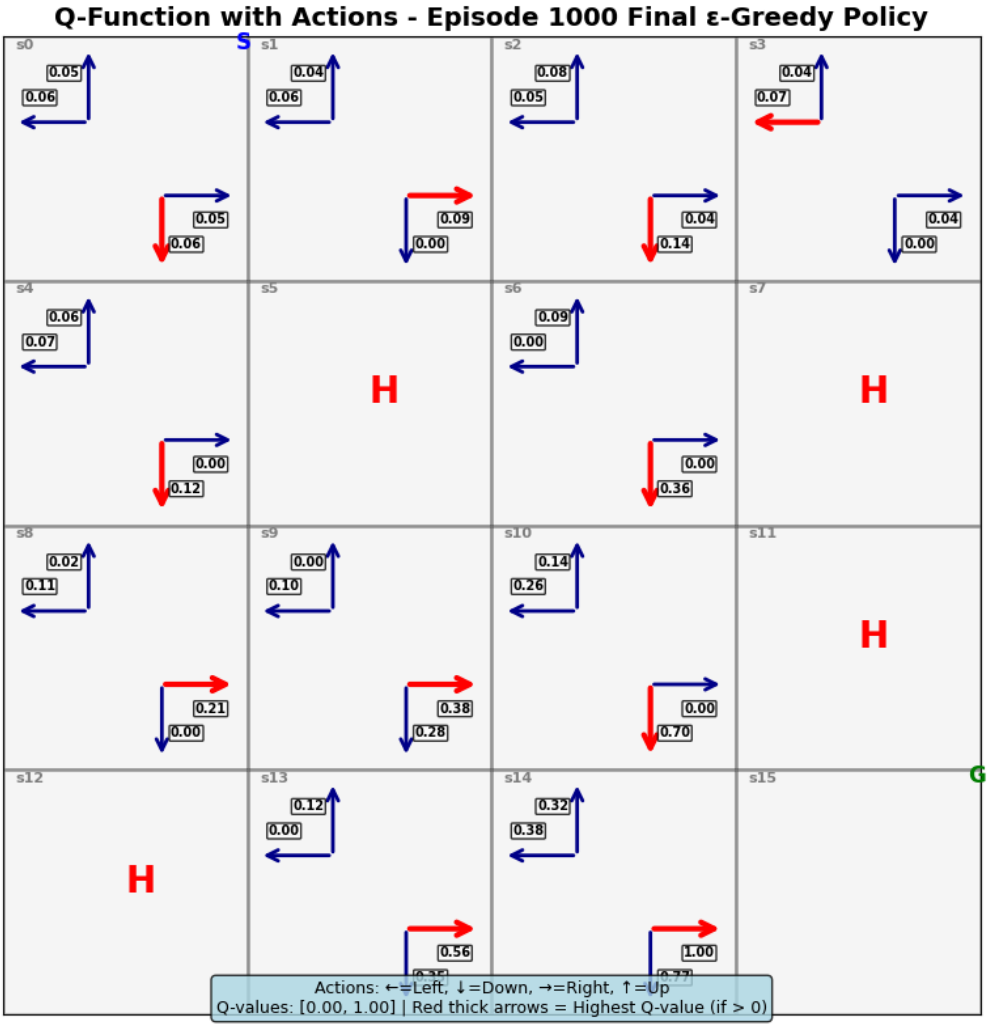
Convergence: This algorithm converges to the best policy among all ε-greedy policies (not fully optimal, but practical).

In [8]:
```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from matplotlib.colors import LinearSegmentedColormap
from IPython.display import clear_output
import time

def monte_carlo_epsilon_greedy_control(env_name='FrozenLake-v1',
num_episodes=5000, gamma=0.95,
                           epsilon=0.1, plot_interval=500, animate
=True, exploring_starts=False):
    """
    Monte Carlo On-Policy Control with ε-Greedy Policies

    Follows pseudocode: Initialize Q, π → Loop episodes with ε-gr
eedy policy

    Args:
        env_name: Gymnasium environment name
        num_episodes: Number of episodes to run
        gamma: Discount factor
        epsilon: Exploration parameter for ε-greedy policy
        plot_interval: Show plot every N episodes
        animate: Whether to show animations
        exploring_starts: If True, use random initial states. If
False, always start from s_0.

    Returns:
        tuple: (Q, policy, returns)
    """
    import gymnasium as gym
    from collections import defaultdict

    max_ep_steps = 50
    headless_env = gym.make(env_name, is_slippery=False, render_m
ode="ansi",
                    max_episode_steps=max_ep_steps, reward_schedul
e=(1, 0, 0))
    render_env = gym.make(env_name, is_slippery=False, render_mod
e="human",
                    max_episode_steps=max_ep_steps, reward_schedul
e=(1, 0, 0))
    n_states, n_actions = render_env.observation_space.n, render_
env.action_space.n

    # Step 1: Initialize Q(s,a) arbitrarily, π(s) as ε-greedy w.
r.t. Q
    Q = np.zeros((n_states, n_actions))
    returns = defaultdict(list)

    def epsilon_greedy_policy(state, Q, epsilon):
        """ε-greedy policy: exploit best action with prob (1-ε),
explore with prob ε"""
        if np.random.random() < epsilon:
            return np.random.randint(0, n_actions)  # Explore: ra
ndom action
```

```
        else:
            return np.argmax(Q[state, :])  # Exploit: greedy acti
on

    es_status = "WITH" if exploring_starts else "WITHOUT"
    print(f"MC ε-Greedy Control {es_status} Exploring Starts:")
    print(f"States: {n_states}, Actions: {n_actions}, Episodes:
{num_episodes}")
    print(f"γ = {gamma}, ε = {epsilon}")

    if animate:
        update_mc_control_plot(Q, 0, "Initial")
        time.sleep(2)

    # Step 2: Repeat for each episode
    for episode in range(1, num_episodes + 1):
        if episode % plot_interval == 0 and animate:
            env = render_env
        else:
            env = headless_env

        # Step 3: Generate episode using ε-greedy policy
        if exploring_starts:
            # Optional: Exploring starts - random s_0 and a_0
            terminal_states = [5, 7, 11, 12, 15]
            valid_starts = [s for s in range(n_states) if s not i
n terminal_states]
            s_0 = np.random.choice(valid_starts)
            a_0 = np.random.randint(0, n_actions)  # Random initi
al action
            env.reset()
            env.unwrapped.s = s_0  # Force start at random state
        else:
            # Standard: Start from default state with ε-greedy ac
tion
            s_0, _ = env.reset()
            a_0 = epsilon_greedy_policy(s_0, Q, epsilon)

        # Generate episode following ε-greedy policy
        episode_data = []
        state, done = s_0, False
        action = a_0

        step_count = 0
        while not done and step_count < max_ep_steps:
            next_state, reward, done, _, _ = env.step(action)
            episode_data.append((int(state), int(action), float(r
eward)))
            state = next_state
            if not done:
                action = epsilon_greedy_policy(state, Q, epsilon)
# ε-greedy action selection
            step_count += 1

        # Step 4: For each step of episode, calculate returns and
update Q
        G = 0
```

```
        for t in range(len(episode_data) - 1, -1, -1):
            state_t, action_t, reward_t_plus_1 = episode_data[t]
            G = gamma * G + reward_t_plus_1

            # First-visit check: unless the pair S_t, A_t appears
in S_0, A_0, ..., S_{t-1}, A_{t-1}
            sa_pairs_before = [(episode_data[i][0], episode_data
[i][1]) for i in range(t)]
            if (state_t, action_t) not in sa_pairs_before:
                # Append G to Returns(S_t, A_t)
                returns[(state_t, action_t)].append(G)
                # Q(S_t, A_t) ← average(Returns(S_t, A_t))
                Q[state_t, action_t] = np.mean(returns[(state_t,
action_t)])
                # π(S_t) ← ε-greedy(Q(S_t, ·)) - implicit in epsi
lon_greedy_policy function

        # Animate progress
        if animate and episode % plot_interval == 0:
            update_mc_control_plot(Q, episode)
            time.sleep(2)
            max_q = np.max(Q)
            mean_q = np.mean(Q[Q > 0]) if np.any(Q > 0) else 0
            print(f"Episode {episode:4d}: Max Q = {max_q:.3f}, Me
an Q = {mean_q:.3f}, "
                  f"SA-pairs explored = {len(returns)}/{n_states*
n_actions}")

    if animate:
        update_mc_control_plot(Q, num_episodes, "Final ε-Greedy P
olicy")
        plt.ioff()
        # Keep the final plot open by not clearing output

    max_q = np.max(Q)
    coverage = len(returns) / (n_states * n_actions) * 100

    print(f"\nConverged! SA-pairs explored: {len(returns)}/{n_sta
tes*n_actions} ({coverage:.1f}%)")
    print(f"Max Q-value: {max_q:.3f}")

    env.close()
    return Q, returns
```

In [9]:
```
# Run MC ε-Greedy Control
Q_eps, returns_eps = monte_carlo_epsilon_greedy_control(
    env_name='FrozenLake-v1',
    num_episodes=1000,
    gamma=0.95,
    epsilon=0.8,    # ε-greedy exploration parameter. Usually aro
und 0.1, but here 0.6 is at least required.
    plot_interval=200,
    animate=True,
    exploring_starts=False  # Standard ε-greedy (set to True to c
ombine with exploring starts)
)
```




```
Converged! SA-pairs explored: 44/64 (68.8%)
Max Q-value: 1.000
```

Monte Carlo Off-Policy Prediction

Motivation: Sometimes we want to learn about one policy (the **target policy** π) while following a different policy (the **behavior policy** b). This enables:

- Learning from historical data generated by a different policy
- Evaluating multiple target policies from a single set of episodes
- Learning optimal policies while maintaining necessary exploration (better exploration than ϵ-greedy)

What it does: Estimates the value function $V^\pi(s)$ or $Q^\pi(s, a)$ of a target policy π using episodes generated by a different behavior policy b.

Importance sampling in off-policy methods

Challenge: Learn value function V^π of target policy π using experience from behavior policy b.

Solution - Importance Sampling: Weight returns by the probability ratio between policies:

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(a_k|s_k)}{b(a_k|s_k)}$$

Key Properties:

- $\rho > 1$: Target policy more likely to take these actions → up-weight return
- $\rho < 1$: Target policy less likely to take these actions → down-weight return
- Rebalances data to reflect what would happen under π

Ordinary Importance Sampling Off-Policy Prediction

Input: Target policy π, behavior policy b
Output: $V^{\pi}(s)$ (value function for all states)

1. Initialize $V(s)$ arbitrarily for all $s \in \mathcal{S}$
2. Initialize $N(s) \leftarrow 0$ for all $s \in \mathcal{S}$
3. Repeat forever:
  4. Generate episode following $b: s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, r_T$
  5. $G \leftarrow 0$
  6. $\rho \leftarrow 1$
  7. For $t = T-1, T-2, \ldots, 0:$
    8. $G \leftarrow \gamma G + r_{t+1}$
    9. $\rho \leftarrow \rho \cdot \dfrac{\pi(a_t|s_t)}{b(a_t|s_t)}$
    10. If $s_t \notin \{s_0, s_1, \ldots, s_{t-1}\}:$ (first-visit condition)
      11. $N(s_t) \leftarrow N(s_t) + 1$
      12. $V(s_t) \leftarrow V(s_t) + \dfrac{\rho \cdot G - V(s_t)}{N(s_t)}$

Key Components:

- $N(s)$: Count of first visits to state s
- ρ: Importance sampling ratio $\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(a_k|s_k)}{b(a_k|s_k)}$
- **Incremental update**: Line 12 computes $V(s) = \frac{1}{N(s)} \sum_{i=1}^{N(s)} \rho_i \cdot G_i$ incrementally
- **First-visit**: Only updates the first time a state is encountered in an episode

Key Properties:

- **Unbiased**: $\mathbb{E}[V(s)] = v_{\pi}(s)$ - the estimate converges to the true value
- **High/Infinite Variance**: Can have extremely large variance, especially for long episodes or small $b(a|s)$
- **Simple averaging**: Each importance-weighted return contributes equally (count-based averaging)

What is wrong with ordinary importance sampling?

The problem is **variance** - it can be extremely high or even infinite! Here's why:

- **Multiplicative ratios grow exponentially**: The importance sampling ratio $\rho = \prod_t \frac{\pi(A_t|S_t)}{b(A_t|S_t)}$ multiplies terms at each step
- **Small differences compound**: If behavior policy b takes even slightly different actions than target policy π, the ratio can explode
- **One bad action ruins everything**: If $b(a|s)$ is very small but $\pi(a|s)$ is large, ρ becomes huge
- **Infinite variance**: If $b(a|s)$ can be arbitrarily close to zero while $\pi(a|s) > 0$, the variance becomes infinite

Example: Suppose b takes an action with probability 0.1, but π would take it with probability 0.9. Just this one action contributes a factor of $\frac{0.9}{0.1} = 9$ to the ratio! After 10 such steps: $9^{10} \approx 3.5$ billion! 💥

The solution: Weighted importance sampling normalizes by the sum of ratios, which dramatically reduces variance and keeps it finite.

Weighted Importance Sampling Off-Policy Prediction

Mathematical Formalization:

For ordinary importance sampling, we compute:

$$V(s) = \frac{\sum_{i=1}^{N(s)} \rho_i \cdot G_i}{N(s)}$$

where $\rho_i = \prod_{k=t}^{T-1} \frac{\pi(a_k|s_k)}{b(a_k|s_k)}$ is the importance sampling ratio for the i-th visit to state s.

Weighted Importance Sampling modifies this to normalize by the sum of ratios:

$$V(s) = \frac{\sum_{i=1}^{N(s)} \rho_i \cdot G_i}{\sum_{i=1}^{N(s)} \rho_i}$$

Off-Policy MC Prediction with Weighted Importance Sampling

The algorithm is identical to ordinary importance sampling, except for **one line** in the update rule:

Input: Target policy π, behavior policy b
Output: $V^{\pi}(s)$ (value function for all states)

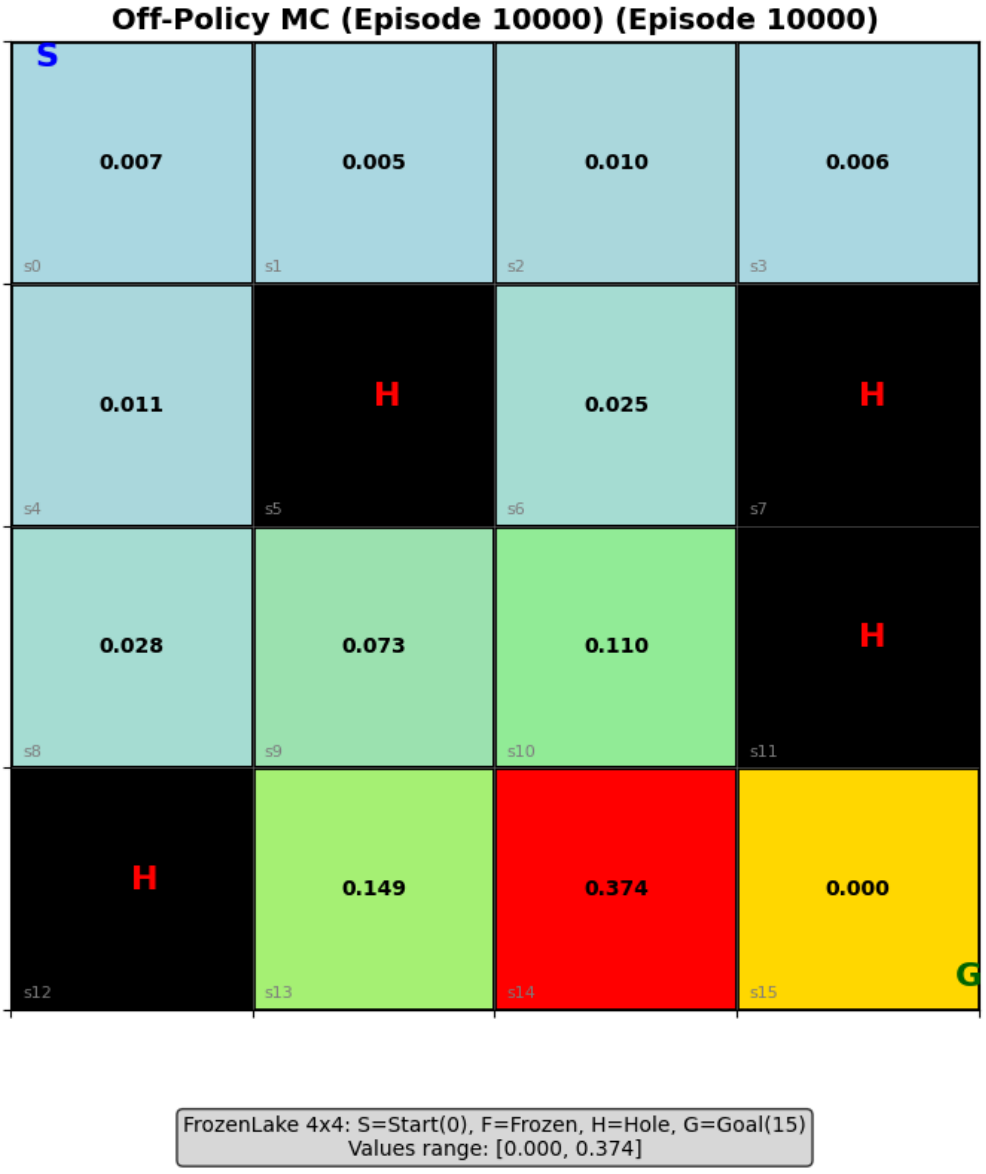
1. Initialize $V(s)$ arbitrarily for all $s \in \mathcal{S}$
2. Initialize $C(s) \leftarrow 0$ for all $s \in \mathcal{S}$
3. Repeat forever:
  4. Generate episode following $b : s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, r_T$
  5. $G \leftarrow 0$
  6. $\rho \leftarrow 1$
  7. For $t = T-1, T-2, \ldots, 0 :$
    8. $G \leftarrow \gamma G + r_{t+1}$
    9. $\rho \leftarrow \rho \cdot \dfrac{\pi(a_t|s_t)}{b(a_t|s_t)}$
    10. If $s_t \notin \{s_0, s_1, \ldots, s_{t-1}\} :$ (first-visit condition)
      11. $C(s_t) \leftarrow C(s_t) + \rho$
      12. $V(s_t) \leftarrow V(s_t) + \dfrac{\rho}{C(s_t)}[G - V(s_t)]$ (weighted IS: uses $C(s)$ instead of $N(s$

In [10]:
```
def _get_policy_prob(policy_fn, state, action, n_actions):
    """Helper: Get probability of action under policy"""
    if policy_fn is None:
        return 1.0 / n_actions
    result = policy_fn(state)
    if isinstance(result, tuple):
        policy_action, prob = result
        return prob if action == policy_action else 0.0
    return 1.0 / n_actions

def _generate_episode(env, policy_fn):
    """Helper: Generate episode following policy"""
    episode_data = []
    state = env.reset()[0]
    done = False
    while not done:
        action = policy_fn(state) if policy_fn else np.random.cho
ice(env.action_space.n)
        if isinstance(action, tuple):
            action = action[0]
        next_state, reward, done, _, _ = env.step(action)
        episode_data.append((state, action, reward))
        state = next_state
    return episode_data

def monte_carlo_off_policy_prediction(env, target_policy_fn, beha
vior_policy_fn,
                                      num_episodes=1000, gamma=0.9
5,
                                      plot_interval=100, show_anim
ation=True):
    """
    Off-Policy MC Prediction with Weighted Importance Sampling

    Learns V^π(s) for target policy π using episodes from behavio
r policy b
    """
    import numpy as np

    # Initialize V(s) and C(s)
    V = np.zeros(env.observation_space.n)
    C = np.zeros(env.observation_space.n)
    n_actions = env.action_space.n

    print(f"Off-Policy MC Prediction: {num_episodes} episodes, γ=
{gamma}")

    if show_animation:
        update_mc_value_function_plot(V, "Initial Value Functio
n", episode=0)

    # Main loop
    for episode in range(num_episodes):
        # Generate episode following behavior policy b
        episode_data = _generate_episode(env, behavior_policy_fn)
```

```
        # Backward pass: G ← 0, W ← 1
        G = 0
        W = 1.0

        for t in range(len(episode_data) - 1, -1, -1):
            state_t, action_t, reward_t_plus_1 = episode_data[t]

            # G ← γG + R_{t+1}
            G = gamma * G + reward_t_plus_1

            # W ← W · π(A_t|S_t) / b(A_t|S_t)
            pi_prob = _get_policy_prob(target_policy_fn, state_t,
action_t, n_actions)
            b_prob = _get_policy_prob(behavior_policy_fn, state_
t, action_t, n_actions)
            W = W * (pi_prob / b_prob) if b_prob > 0 else 0

            if W == 0:
                break

            # First-visit check
            if state_t not in [episode_data[i][0] for i in range
(t)]:
                # C(S_t) ← C(S_t) + W
                C[state_t] += W

                # V(S_t) ← V(S_t) + (W/C(S_t)) * [G - V(S_t)]
                if C[state_t] > 0:
                    V[state_t] += (W / C[state_t]) * (G - V[state
_t])

        # Visualization
        if show_animation and (episode % plot_interval == 0 or ep
isode == num_episodes - 1):
            if episode % plot_interval == 0:
                print(f"Episode {episode}/{num_episodes}")
            update_mc_value_function_plot(V, f"Off-Policy MC (Epi
sode {episode+1})", episode=episode+1)

    print(f"\nCompleted {num_episodes} episodes")
    print("Value Function V^π(s):")
    for s in range(len(V)):
        print(f"  State {s}: V={V[s]:.4f}, C={C[s]:.2f}")

    return V, C
```

In [11]:

```
# Run Off-Policy MC Prediction
import gymnasium as gym

# Create environment
env = gym.make('FrozenLake-v1', is_slippery=False, render_mode="a
nsi",
               max_episode_steps=50, reward_schedule=(1, 0, 0))

# Define target policy π (what we want to evaluate) - e.g., alway
s go right
def target_policy(state):
    # Simple deterministic policy: right (1) when possible
    return 1  # Always go right

# Define behavior policy b (exploratory) - random policy
def behavior_policy(state):
    return np.random.choice(env.action_space.n)

# Run off-policy prediction
V, C = monte_carlo_off_policy_prediction(
    env=env,
    target_policy_fn=target_policy,
    behavior_policy_fn=behavior_policy,
    num_episodes=10000,
    gamma=0.95,
    plot_interval=100,
    show_animation=True
)
```




```
Completed 10000 episodes
Value Function V^π(s):
  State 0: V=0.0073, C=10000.00
  State 1: V=0.0047, C=6198.00
  State 2: V=0.0101, C=2633.00
  State 3: V=0.0064, C=1152.00
  State 4: V=0.0108, C=6247.00
  State 5: V=0.0000, C=0.00
  State 6: V=0.0246, C=1359.00
  State 7: V=0.0000, C=0.00
  State 8: V=0.0279, C=2578.00
  State 9: V=0.0730, C=1150.00
  State 10: V=0.1099, C=726.00
  State 11: V=0.0000, C=0.00
  State 12: V=0.0000, C=0.00
  State 13: V=0.1491, C=420.00
  State 14: V=0.3736, C=327.00
  State 15: V=0.0000, C=0.00
```

Monte Carlo Off-Policy Control

Motivation: We want to find an optimal policy while using a different exploratory policy to generate data. This separation allows:

- The **target policy** to be deterministic and greedy (potentially optimal)
- The **behavior policy** to be exploratory (ensuring sufficient data coverage)
- Learning from data generated by any exploratory policy, including human experts or other agents

What it does: Discovers an optimal policy by iteratively improving a target policy using episodes generated by a different behavior policy.

How it works:

1. **Policy separation**:
   - **Target policy** $\pi(s)$: The policy being learned and improved (deterministic, greedy w.r.t. Q)
   - **Behavior policy** $b(a|s)$: The policy generating episodes (exploratory, e.g., ϵ-soft)
2. **Off-policy Generalized Policy Iteration (GPI) with importance sampling**:
   - **Evaluation**: Estimate $Q^\pi(s, a)$ using importance-weighted returns from behavior policy episodes
   - **Improvement**: Update target policy to be greedy: $\pi(s) = \arg\max_a Q(s, a)$
3. **Incremental updates**: Use weighted importance sampling for Q-value updates:

   $$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}[G - Q(S_t, A_t)]$$

   where $W = \prod_{k=t+1}^{T-1} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}$ and $C(s, a)$ tracks cumulative weights
4. **Early termination**: Updates stop when behavior policy deviates from target policy (when $\pi(A_t|S_t) \neq b(A_t|S_t)$)

Off-Policy MC Control Pseudocode

Algorithm: Off-Policy MC Control for estimating $\pi \approx \pi^*$

$$
\begin{aligned}
&\textbf{Initialize, for all } s \in \mathcal{S}, a \in \mathcal{A}(s)\textbf{:}\\
&\quad Q(s,a) \in \mathbb{R} \text{ (arbitrarily)}\\
&\quad C(s,a) \leftarrow 0\\
&\quad \pi(s) \leftarrow \arg\max_a Q(s,a) \text{ (with ties broken consistently)}\\
\\
&\textbf{Loop forever (for each episode):}\\
&\quad b \leftarrow \text{any soft policy}\\
&\quad \text{Generate an episode using } b : s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T\\
&\quad G \leftarrow 0\\
&\quad \rho \leftarrow 1\\
&\quad \textbf{Loop for each step of episode, } t = T-1, T-2, \ldots, 0\textbf{:}\\
&\quad\quad G \leftarrow \gamma G + r_{t+1}\\
&\quad\quad C(s_t, a_t) \leftarrow C(s_t, a_t) + \rho\\
&\quad\quad Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \frac{\rho}{C(s_t, a_t)}[G - Q(s_t, a_t)]\\
&\quad\quad \pi(s_t) \leftarrow \arg\max_a Q(s_t, a) \text{ (with ties broken consistently)}\\
&\quad\quad \textbf{If } a_t \neq \pi(s_t) \textbf{ then exit inner Loop } \text{(proceed to next episode)}\\
&\quad\quad \rho \leftarrow \rho \cdot \frac{1}{b(a_t|s_t)}
\end{aligned}
$$

Key Components:

- $\pi(s)$: Target policy (deterministic, greedy w.r.t. Q)
- $b(a|s)$: Behavior policy (soft/exploratory, e.g., ϵ-soft)
- $C(s,a)$: Cumulative sum of importance sampling weights
- ρ: Importance sampling ratio (simplified since π is deterministic: $\pi(a_t|s_t) \in \{0,1\}$)
- **Early exit**: When $a_t \neq \pi(s_t)$, the importance ratio becomes 0, so we stop updating

Why this works:

Since π is deterministic (greedy), $\pi(a_t|s_t) = 1$ if $a_t = \pi(s_t)$ and 0 otherwise. Therefore:

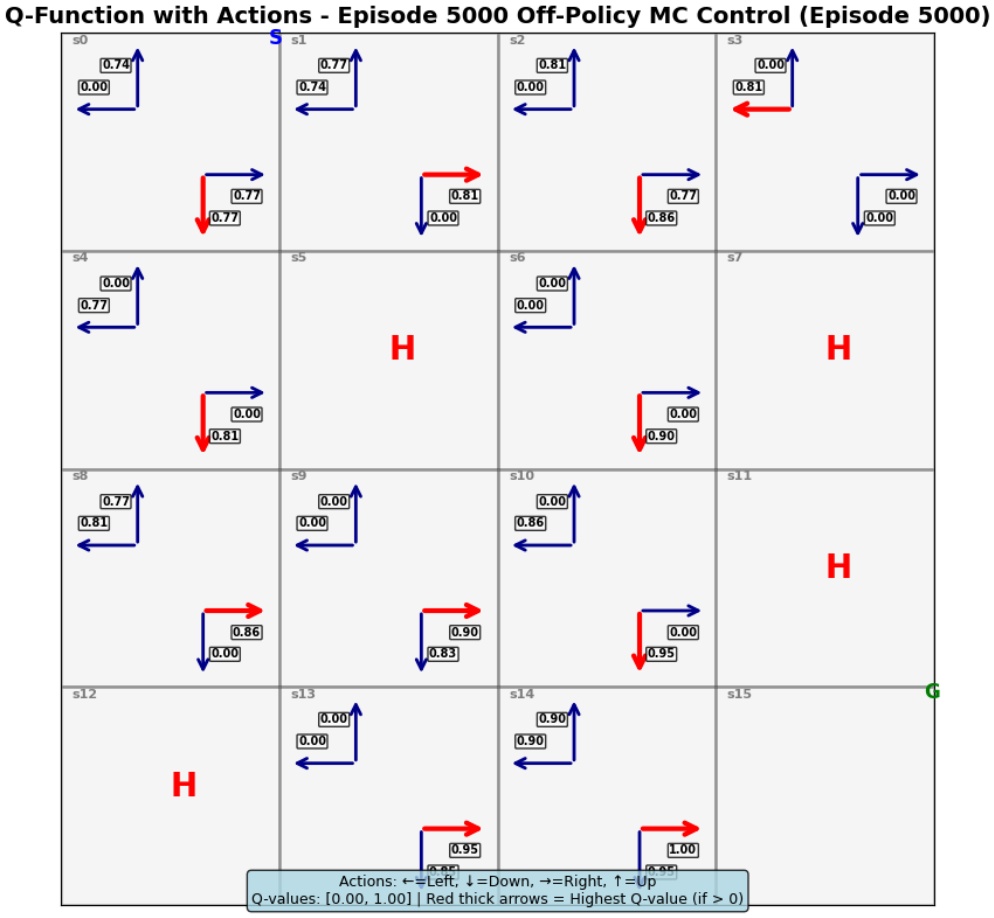
- If $a_t = \pi(s_t)$: $\rho \leftarrow \rho \cdot \frac{1}{b(a_t|s_t)}$ (continue)
- If $a_t \neq \pi(s_t)$: $\rho \to 0$ (exit loop immediately)

In [12]:
```
def monte_carlo_off_policy_control(env, behavior_policy_fn, epsil
on=0.1,
                                   num_episodes=1000, gamma=0.95,
                                   plot_interval=100, show_animat
ion=True):
    """
    Off-Policy MC Control for estimating π ≈ π*

    Learns optimal policy π using episodes from exploratory behav
ior policy b
    """
    import numpy as np

    n_states = env.observation_space.n
    n_actions = env.action_space.n

    # Initialize Q(s,a) arbitrarily and C(s,a) = 0
    Q = np.zeros((n_states, n_actions))
    C = np.zeros((n_states, n_actions))

    # Initialize target policy π (greedy w.r.t. Q)
    pi = np.argmax(Q, axis=1)

    print(f"Off-Policy MC Control: {num_episodes} episodes, γ={ga
mma}")

    if show_animation:
        update_mc_control_plot(Q, 0, "Initial Q and Policy")

    # Main loop: for each episode
    for episode in range(num_episodes):
        # Generate episode using behavior policy b
        episode_data = _generate_episode(env, behavior_policy_fn)

        # G ← 0, W ← 1
        G = 0
        W = 1.0

        # Loop for each step of episode, t = T-1, T-2, ..., 0
        for t in range(len(episode_data) - 1, -1, -1):
            state_t, action_t, reward_t_plus_1 = episode_data[t]

            # G ← γG + R_{t+1}
            G = gamma * G + reward_t_plus_1

            # C(S_t, A_t) ← C(S_t, A_t) + W
            C[state_t, action_t] += W

            # Q(S_t, A_t) ← Q(S_t, A_t) + (W/C(S_t,A_t)) * [G - Q
(S_t, A_t)]
            if C[state_t, action_t] > 0:
                Q[state_t, action_t] += (W / C[state_t, action_
t]) * (G - Q[state_t, action_t])

            # π(S_t) ← argmax_a Q(S_t, a) (with ties broken consi
stently)
```

```
            pi[state_t] = np.argmax(Q[state_t, :])

            # If A_t ≠ π(S_t) then exit inner loop
            if action_t != pi[state_t]:
                break

            # W ← W · 1/b(A_t|S_t)
            b_prob = _get_policy_prob(behavior_policy_fn, state_
t, action_t, n_actions)
            W = W / b_prob if b_prob > 0 else 0

            if W == 0:
                break

        # Visualization
        if show_animation and (episode % plot_interval == 0 or ep
isode == num_episodes - 1):
            if episode % plot_interval == 0:
                print(f"Episode {episode}/{num_episodes}")
            update_mc_control_plot(Q, episode+1, f"Off-Policy MC
Control (Episode {episode+1})")
            time.sleep(5)

    print(f"\nCompleted {num_episodes} episodes")
    print("Learned Policy π(s):")
    action_names = ['←', '↓', '→', '↑']
    for s in range(n_states):
        print(f"  State {s}: π={action_names[pi[s]]} (Q={Q[s, pi
[s]]:.3f})")

    return Q, pi, C
```

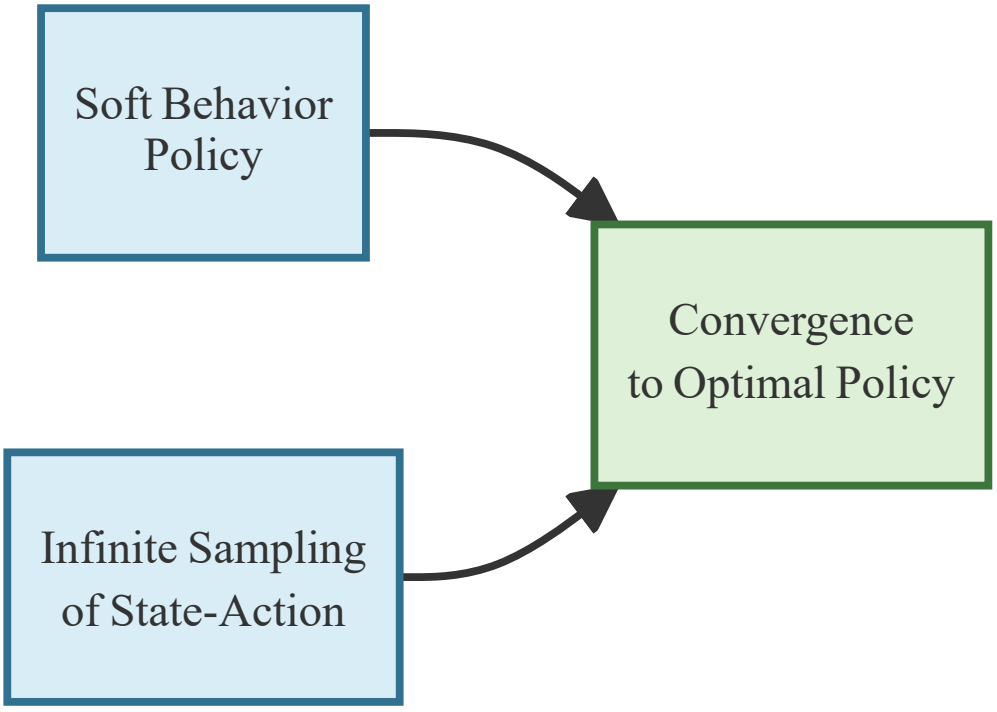
In [13]:

```
# Run Off-Policy MC Control
import gymnasium as gym

# Create environment
env = gym.make('FrozenLake-v1', is_slippery=False, render_mode="a
nsi",
               max_episode_steps=50, reward_schedule=(1, 0, 0))

# Define behavior policy b (ε-soft, exploratory)
def behavior_policy(state):
    # For simplicity, return random action (fully exploratory)
    return np.random.choice(env.action_space.n)

# Run off-policy control
Q, pi, C = monte_carlo_off_policy_control(
    env=env,
    behavior_policy_fn=behavior_policy,
    epsilon=0.3,
    num_episodes=5000,
    gamma=0.95,
    plot_interval=500,
    show_animation=True
)

print("\nOptimal Policy found!")
print("Policy visualization (4x4 grid):")
action_symbols = ['←', '↓', '→', '↑']
for row in range(4):
    print(" ".join([action_symbols[pi[row*4 + col]] for col in ra
nge(4)]))
```




```
Completed 5000 episodes
Learned Policy π(s):
  State 0: π=↓ (Q=0.774)
  State 1: π=→ (Q=0.815)
  State 2: π=↓ (Q=0.857)
  State 3: π=← (Q=0.815)
  State 4: π=↓ (Q=0.815)
  State 5: π=← (Q=0.000)
  State 6: π=↓ (Q=0.902)
  State 7: π=← (Q=0.000)
  State 8: π=→ (Q=0.857)
  State 9: π=→ (Q=0.902)
  State 10: π=↓ (Q=0.950)
  State 11: π=← (Q=0.000)
  State 12: π=← (Q=0.000)
  State 13: π=→ (Q=0.950)
  State 14: π=→ (Q=1.000)
  State 15: π=← (Q=0.000)

Optimal Policy found!
Policy visualization (4x4 grid):
↓ → ↓ ←
↓ ← ↓ ←
→ → ↓ ←
← → → ←
```

Convergence of off-policy control methods



Convergence with soft behavior policy

- Off-policy MC control
- Converges to optimal policy
- Soft behavior policy
- Infinite sampling of state-action pairs

Dependence on behavior policy

- Convergence depends on behavior policy
- All actions must be sampled
- Unselected actions → non-convergence

Practical considerations for off-policy control

Incremental implementation for efficiency

- Incremental implementation
- Maintains cumulative weights
- Updates action-value estimates per episode
- Reduces memory usage

Behavior policy flexibility

- Behavior policy flexibility
- Can change between/within episodes
- Must remain soft
- Ensures exploration of all state-action pairs

Summary: Monte Carlo Methods

Four Main Algorithms

Algorithm	Type	Key Features	Update Rule
MC Prediction (On-Policy)	Evaluation	Estimates $V^\pi(s)$ by averaging returns	$V(s) \leftarrow V(s) + \frac{1}{N(s)}[G - V(s)]$
MC ES Control	Control	Uses exploring starts (s_0, a_0) random	$Q(s,a) \leftarrow Q(s,a) + \alpha[G - Q(s,a)]$
MC Off-Policy Prediction	Evaluation	Learns V^π from behavior policy b	$V(s) \leftarrow V(s) + \frac{W}{C(s)}[G - V(s)]$
MC Off-Policy Control	Control	Learns π^* from exploratory b	$Q(s,a) \leftarrow Q(s,a) + \frac{W}{C(s,a)}[G - Q(s,a)]$

Key Variants and Techniques

1. First-Visit vs Every-Visit MC

- **First-visit**: Update only on first occurrence of state in episode
- **Every-visit**: Update on every occurrence (higher variance, faster convergence in practice)

2. Incremental Updates

- Avoids tracking full history of returns - updates value estimates incrementally
- **Sample averaging**: $\alpha = \frac{1}{N(s)}$ - true Monte Carlo
- **Constant step-size**: $\alpha = \text{const}$ - tracks non-stationary environments

3. Importance Sampling

- **Ordinary IS**: $V(s) = \frac{1}{N(s)} \sum \rho_i G_i$ - high variance
- **Weighted IS**: $V(s) = \frac{\sum \rho_i G_i}{\sum \rho_i}$ - less variance ✅

4. Exploring Starts (ES)

- Ensures all (s, a) pairs visited by randomizing initial state and action

- Eliminates need for ϵ-soft policies in control
- Practical limitation: not always feasible in real applications

Common Characteristics of All MC Methods

✅ **Model-free**: No need for environment dynamics $p(s', r|s, a)$
✅ **Complete episodes**: Must wait until termination to update
✅ **Sample-based**: Learn from experience, not full distributions
✅ **Generalized Policy Iteration**: Alternate evaluation ↔ improvement

Critical Weaknesses of Monte Carlo Methods

1. Episode Completion Requirement 🕐

- **Must wait until episode ends** to compute returns G_t
- **No online learning**: Cannot update during episode
- **Problem**: Unsuitable for continuing tasks (no terminal states)
- **Problem**: Slow for long episodes (e.g., 1000+ steps)

2. High Variance 📊

- Returns $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$ accumulate noise
- Each reward adds variance: $\mathrm{Var}(G_t) = \sum_{k=0}^{\infty} \gamma^{2k} \mathrm{Var}(R_{t+k+1})$
- **Problem**: Requires many episodes for accurate estimates
- **Off-policy**: Importance sampling ratios $\rho = \prod \frac{\pi}{b}$ explode exponentially

3. Slow Learning 🐌

- **Early states**: Updated only from episode tails (off-policy control)
- **Credit assignment**: Must trace back through entire episode
- **Problem**: Takes thousands of episodes to converge

4. No Bootstrapping 🚫

- **Cannot leverage existing estimates** of future states
- Each return is computed independently from scratch
- **Problem**: Ignores value function knowledge accumulated so far

Example: Why MC is Slow

Consider FrozenLake with 100-step episodes:

- **MC**: Must complete all 100 steps before updating $V(s_0)$
- **Issue**: If goal is at step 100, variance from 99 intermediate rewards
- **Issue**: 1 bad action at step 50 → entire return is corrupted

Preview: Temporal-Difference (TD) Learning

How TD Overcomes MC Weaknesses

MC Weakness	TD Solution
❌ Requires episode completion	✅ **Online updates** after every step
❌ High variance (full returns)	✅ **Bootstrapping**: $V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$
❌ Slow learning (long episodes)	✅ **Immediate updates** using 1-step lookahead
❌ Cannot handle continuing tasks	✅ **Works without terminal states**
❌ No use of existing estimates	✅ **Leverages current** $V(s')$ for faster learning

Central Idea: Bootstrapping

Monte Carlo: Wait for actual return

$$V(s_t) \leftarrow V(s_t) + \alpha[G_t - V(s_t)]$$

Temporal-Difference: Use estimate of return

$$V(s_t) \leftarrow V(s_t) + \alpha[\underbrace{r_{t+1} + \gamma V(s_{t+1})}_{\text{TD target}} - V(s_t)]$$

Key insight: $r_{t+1} + \gamma V(s_{t+1})$ is a **biased but lower-variance** estimate of G_t

TD Advantages

1. **Learn before episode ends** - update immediately after each transition
2. **Learn from incomplete sequences** - continuing tasks, interrupted episodes
3. **Lower variance** - single reward r_{t+1} vs full return G_t
4. **Faster convergence** - leverages existing value estimates (bootstrapping)
5. **More efficient** - especially for long or infinite episodes

Characteristics of TD methods (combine the best of both worlds):

- **Model-free** like MC (learn from experience)
- **Bootstrapping** like DP (use value estimates)
- **Online** unlike both MC and DP